CS 0007: Introduction to Java Lecture 17

Nathan Ong
University of Pittsburgh
November 8, 2016

Main Portions of OOP

- Building a class
- Determining inter-class relationships

Class Components

- Class-level/Instance Variables
- Methods
- Constructor(s)

Inter-class Relationships

- Building a single class is useless. It needs to be used in context.
- There are two main relationships between classes
  - Sub/Super class
  - Ownership

Subclasses

- A subclass of a class is a more specific type of a class
- Also known as the child of a class
- For example, a tree would be a subclass of plant
- Subclasses are meant to expand upon the behavior of their more general parent class
- You can have several subclasses

Superclasses

- On the converse, a superclass is a more abstract and general version of a class
- Also known as the parent of a class
- Plant is the superclass of Tree
- You can only have one superclass

Declaring a Class to be a Subtype

- In order to declare a class's parent, use the keyword `extends`
- (Classes that do not have a parent class implicitly extend the Object class)

Inheritance

- A subclass inherits all non-private fields and methods
- Accessing these fields and methods requires the keyword `super`.
- (Note: you may also be able to access private fields with the parent class's getters and setters, which may be safer)

Restricting Access

- What if you want a field or method to be inherited, but still do not necessarily want it to be accessed by anyone?
- Use the keyword `protected`
- Remember, releasing your access to these fields and methods should only be done when necessary

Regarding Constructors

- Constructors cannot be inherited, even though they are public
- However, you can, and should, use them from the subclass's constructor, to ensure complete initialization
- You can call the parent's constructor by using the keyword **`super`**

Calling Superclass Constructor

- Unlike calling an overloaded constructor of the same class, calling a superclass constructor does not need to be the only line of code
- However, it must still be the first line of code in the constructor

Example

```
public class Vehicle
{
  private final int numWheels;
  …
  public Vehicle(int numWheels)
  {
  this.numWheels = numWheels;
  …
  }//end function(double)
  …
}//End class Vehicle
```

Example

```
public class Car extends Vehicle
{
 private int numPassengers;
 private double gasRemaining;
 …
 public Car(int numPassengers, double gasRemaining)
 {
 super(4);
 this.numPassengers = numPassengers;
 this.gasRemaining = gasRemaining;
 }//end constructor(int,double)
 …
}//End class Plant
```

What If the Superclass's Method Definition Does Not Fit the Class's?

- We can override the definition by using the exact same function header
- The line directly above function header must have "`@Override`" only
- As an example, our Car class should probably use the original "move" method, rather than creating an entirely new "drive" method that does something similar

Override

```
public class Car extends Vehicle
{
  private int numPassengers;
  private double gasRemaining;
  …
  @Override
  public boolean move(double dist)
  {
  return super.move(dist*10.0) &&
      (this.gasRemaining > 0);
  }//end function(double)
  …
}//End class Plant
```

Overriding

- I can still reference the original method using the `super` keyword
- Unless your overridden method totally discards the functionality of the superclass's method, you should call the superclass's method at least once

Note

- You cannot override constructors

Ownership

- Ownership is simply one class using another as a property
- A Car may have an Engine, a Radio, and an Owner, depending on what you need
- A Car’s superclass and subclasses certainly are not Engine, Radio, and Owner

Using the Classes

```
public class Garage
{
 private Vehicle[] vehicles;
 …
 public Garage(int capacity)
 {
 this.vehicles = new Vehicle[capacity];
 for(int i = 0; i < vehicles.length; i++)
 {
 vehicles[i] = null;
 }
 …
 }//end constructor(int)
 …
}//End class Garage
```

Using the Classes

```
public class Garage
{
 …
 public boolean addVehicle(Vehicle vehicle)
 {
 //Note: there is a better way!
 for(int i = 0; i < vehicles.length; i++)
 {
 if(vehicles[i] == null)
 {
 vehicles[i] = vehicle;
 return true;
 }
 }
 return false;
 }//end method(Vehicle)
}//End class Garage
```

QUESTIONS?

Modeling a Problem

"Joe's Automotive Shop services foreign cars, and specializes in servicing cars made by Mercedes, Porsche, and BMW. When a customer brings a car to the shop, the manager gets the customer's name, address, and telephone number. Then the manager determines the make, model, and year of the car, and gives the customer a service quote. The service quote shows the estimated parts charges, estimated labor charges, sales tax, and total estimated charges."

Source: Starting Out with Java: From Control Structures through Objects by Tony Gaddis

Find the Nouns

"Joe's Automotive Shop services foreign cars, and specializes in servicing cars made by Mercedes, Porsche, and BMW. When a customer brings a car to the shop, the manager gets the customer's name, address, and telephone number. Then the manager determines the make, model, and year of the car, and gives the customer a service quote. The service quote shows the estimated parts charges, estimated labor charges, sales tax, and total estimated charges."

Coalescing

address
BMW
car
quote
cars
customer
estimated labor charges
charges
estimated parts charges

foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name

Porsche
sales tax
service
shop
telephone number
total estimated
year

Noun Out of Scope

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Subtypes

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Whether these subtypes are specific objects (instances) or more specific types of cars (subclasses) will depend on your needs. In this scenario, there is no need to represent them as classes.

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe’s Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Simple properties can be composed of primitives or preexisting object definitions, and fall under a particular object class.

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A car had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A car had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A customer had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

A customer had three simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated
year

A service quote had four simple properties in our scenario. What types would they be?

Simple Properties

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated
year

A service quote had four simple properties in our scenario. What types would they be?

Who Owns Everything?

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

Our scenario revolves around the automotive shop, so this will hold the main method and will be the overarching class.

Code on the board!